

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Sprawozdanie z realizacji projektu

RAG – pytania i odpowiedzi w oparciu o bazę dokumentów

Jakub Bagiński, Maciej Borkowski

Numer albumu 331357, 331667

WARSZAWA 2026

Spis treści

1. Wstęp	3
1.1. Opis problemu	3
1.2. Ilustracja problemu	3
1.3. Motywacja	3
1.4. Wkład własny	3
2. Przegląd literatury	5
3. Opis rozwiązania	6
3.1. Architektura ogólna	6
3.2. Zbiór danych	6
3.3. Przygotowanie bazy dokumentów	6
3.4. Wariant A - bez RAG	6
3.5. Wariant B - RAG z BM25 (sparse retrieval)	6
3.6. Wariant C - RAG z gęstym wyszukiwaniem (dense retrieval)	6
3.7. Wariant D - RAG z gęstym wyszukiwaniem i re-rankingiem	7
3.8. Model generatywny	7
3.9. Biblioteki i narzędzia	7
4. Wyniki ewaluacji eksperymentalnej	8
4.1. Metryki ewaluacyjne	8
Jakość generowania	8
Jakość wyszukiwania	8
4.2. Eksperyment 1 - Wpływ RAG na jakość odpowiedzi	8
4.3. Eksperyment 2 - Jakość wyszukiwania dokumentów	8
5. Podsumowanie	10
5.1. Osiągnięte rezultaty	10
5.2. Analiza krytyczna	10
5.3. Dalsze prace	11
Bibliografia	12

1. Wstęp

1.1. Opis problemu

Współczesne duże modele językowe (LLM) cechują się imponującymi zdolnościami generatywnymi, jednak posiadają istotne ograniczenia: ich wiedza jest zamrożona w momencie trenowania, a przy odpowiadaniu na szczegółowe pytania faktograficzne mogą produkować halucynacje - przekonująco brzmiące, lecz błędne odpowiedzi. Szczególnie widać to w zadaniach *question answering* (QA), gdzie model powinien oprzeć swoją odpowiedź na konkretnym fragmencie tekstu.

Retrieval-Augmented Generation (RAG) to podejście, które łączy dwa etapy: wyszukiwanie (*retrieval*) - odnalezienie w bazie dokumentów fragmentów trafnych dla danego pytania, oraz generowanie (*generation*) - sformułowanie odpowiedzi przez model języka na podstawie dostarczonych fragmentów. Dzięki temu model ma dostęp do aktualnej, weryfikowalnej wiedzy bez potrzeby ponownego trenowania.

1.2. Ilustracja problemu

Przykładowe pytanie z datasetu SQuAD: „*What is the population of the Commonwealth?*”

- **Oczekiwana odpowiedź (ground truth):** *2.2 billion*
- **Odpowiedź modelu bez RAG (Wariant A):** model może udzielić odpowiedzi ogólnej lub nieprecyzyjnej, bez gwarancji gruntowania w tekście źródłowym.
- **Odpowiedź modelu z RAG (Wariant D):** model otrzymuje jako kontekst odpowiedni akapit Wikipedii i formułuje zwięzłą, poprawną odpowiedź.

1.3. Motywacja

Systemy QA oparte na RAG mają szerokie zastosowanie praktyczne: asystenci dokumentacyjni, chatboty obsługi klienta, narzędzia do przeszukiwania baz wiedzy firmowej. Porównanie metod wyszukiwania (leksykalnego i semantycznego) oraz efektu re-rankingu pozwala zrozumieć, które składniki pipeline’u RAG mają największy wpływ na jakość końcowych odpowiedzi.

1.4. Wkład własny

W ramach projektu wykonano:

- Wczytanie i przetworzenie zbioru danych rajpurkar/squad (87 599 przykładów ze zbioru treningowego).
- Podzielenie dokumentów na fragmenty (*chunks*) o rozmiarze 256 tokenów z zakładką 50 tokenów, z filtracją zbyt krótkich fragmentów.

- Zbudowanie trzech systemów wyszukiwania: BM25 (sparse), ChromaDB z embeddingami bge-small-en-v1.5 (dense) oraz dense z re-rankiem cross-encoderem ms-marco-MiniLM-L-6-v2.
- Integrację modelu generatywnego Meta-Llama-3-8B-Instruct (kwantyzacja 4-bit NF4) przez bibliotekę LlamaIndex.
- Implementację czterech wariantów systemu (A-D) z jednolitym interfejsem `ask(question, variant)`.
- Ewaluację jakości wyszukiwania (Recall@k) i generowania (ROUGE-L, chrF) na 100 losowych próbkach.

2. Przegląd literatury

System zaimplementowany w projekcie opiera się na kilku fundamentalnych koncepcjach opisanych w literaturze naukowej.

Retrieval-Augmented Generation. Podstawowe podejście RAG zostało zaproponowane przez Lewisa i in. [1], gdzie autorzy łączą gęste wyszukiwanie z modelem generatywnym seq2seq w zadaniach open-domain QA. Wykazali, że gruntowanie odpowiedzi na pobranych dokumentach znacząco redukuje halucynacje i poprawia faktyczność.

Gęste wyszukiwanie (Dense Retrieval). Karpukhin i in. [2] zaproponowali Dense Passage Retrieval (DPR) – metodę kodowania pytań i akapitów w przestrzeni wektorowej za pomocą dwóch odrębnych enkoderów BERT, umożliwiającą wyszukiwanie najbardziej semantycznie zbliżonych fragmentów. W projekcie zastosowano model `bge-small-en-v1.5` z rodziny BGE [3], który jest lżejszą, ale wydajną alternatywą dla DPR.

BM25 i wyszukiwanie leksykalne. Robertson i Zaragoza [4] opisali funkcję rankingową BM25 opartą na modelu probabilistycznym TF-IDF. BM25 pozostaje silnym punktem odniesienia w wyszukiwaniu informacji i dobrze sprawdza się przy pytaniach zawierających rzadkie, charakterystyczne słowa kluczowe.

Re-ranking z cross-enkoderem. Nogueira i Cho [5] wprowadzili dwuetapowy pipeline: taniego bi-enkodera do szybkiego wstępnego pobierania dokumentów oraz kosztowniejszego cross-enkodera do precyzyjnego re-rankingu. W projekcie zastosowano cross-enkoder `ms-marco-MiniLM-L-6-v2` [6] wytrenowany na zbiorze MS MARCO.

Dataset SQuAD. Rajpurkar i in. [7] opublikowali zbiór SQuAD zawierający ponad 100 000 par pytanie–odpowiedź opartych na artykułach Wikipedii. Każda odpowiedź jest wycinkiem tekstu z danego akapitu, co pozwala na precyzyjną ocenę jakości zarówno wyszukiwania, jak i generowania.

Metryki ewaluacyjne. ROUGE [8] i chrF [9] to powszechnie stosowane metryki do oceny jakości generowanych tekstów przez porównanie z referencją na poziomie n-gramów odpowiednio słów i znaków.

3. Opis rozwiązania

3.1. Architektura ogólna

System składa się z czterech głównych komponentów: modułu przygotowania danych, trzech wariantów retrievera, opcjonalnego re-rankera oraz generatora. Wszystkie warianty korzystają z tego samego modelu generatywnego i tego samego promptu, co zapewnia kontrolowaną podstawę do porównań.

3.2. Zbiór danych

Wykorzystano zbiór **SQuAD** (rajpurkar/squad) z Hugging Face - 87 599 przykładów ze zbioru treningowego. Każdy przykład zawiera: tytuł artykułu Wikipedii, akapit kontekstu, pytanie oraz odpowiedź (wycinak z tekstu). Do ewaluacji losowo wybrano 100 próbek (seed=42).

3.3. Przygotowanie bazy dokumentów

- **Rozmiar chunku:** 256 tokenów
- **Zakładka (*overlap*):** 50 tokenów
- Odfiltrowano chunki krótsze niż 40 słów oraz artykuły krótsze niż 200 znaków.
- Usunięto duplikaty na podstawie treści.

Każdy chunk zachowuje metadane: identyfikator pytania, tytuł artykułu, oryginalne pytanie i wzorcową odpowiedź - co umożliwi ewaluację Recall@k przez sprawdzenie, czy chunk powiązany z danym pytaniem znalazł się w top-k wynikach.

3.4. Wariant A - bez RAG

Model Llama 3 8B Instruct jest pytany bezpośrednio, bez dostarczenia kontekstu. Stanowi punkt odniesienia dla pozostałych wariantów.

3.5. Wariant B - RAG z BM25 (sparse retrieval)

Teksty chunków tokenizowane są za pomocą modelu spaCy (en_core_web_sm) z lematyzacją i usunięciem stop-słów. Indeks BM25 (implementacja BM25Okapi z biblioteki rank-bm25) przechowuje przetokenizowane chunki. Dla każdego pytania obliczane są wyniki BM25, a top- k (domyślnie $k = 20$) chunków jest przekazywanych do generatora jako kontekst.

3.6. Wariant C - RAG z gęstym wyszukiwaniem (dense retrieval)

Chunki są indeksowane w bazie wektorowej **ChromaDB** przy użyciu embeddingów BAAI/bge-small-en-v1.5. Wyszukiwanie odbywa się przez podobieństwo kosinusowe (przybliżony nearest-neighbor). Top- k chunków (domyślnie $k = 20$) trafia do generatora.

3.7. Wariant D - RAG z gęstym wyszukiwaniem i re-rankingiem

Wariant D rozbudowuje Wariant C o dodatkowy krok re-rankingu. Po pobraniu top-20 chunków z ChromaDB, cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2 ocenia każdą parę (pytanie, chunk) i re-rankuje wyniki. Do generatora przekazywane jest top-10 chunków po re-rankingu.

3.8. Model generatywny

Jako model generatywny zastosowano **Llama-3-8B-Instruct** (meta-llama/Meta-Llama-3-8B-Instruct) załadowany z kwantyzacją 4-bit NF4 (biblioteka bitsandbytes) w celu redukcji zużycia pamięci GPU. Generacja jest deterministyczna (do_sample=False, max_new_tokens=128).

Zastosowany szablon promptu:

You are a question answering assistant.

Answer ONLY using the provided context.

If the answer cannot be found in the context,
respond exactly:

I could not find the answer.

Context:

{context}

Question:

{question}

Answer:

3.9. Biblioteki i narzędzia

Komponent	Biblioteka / Model
Orkiestrator RAG	LlamaIndex
Embeddingi	BAAI/bge-small-en-v1.5 (HuggingFaceEmbeddings)
Baza wektorowa	ChromaDB
Sparse retrieval	BM25Okapi (rank-bm25), spaCy
Re-ranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Generator	Llama 3 8B Instruct (transformers, 4-bit NF4)
Ewaluacja	evaluate (ROUGE, chrF)
Dataset	rajpurkar/squad (Hugging Face Datasets)

4. Wyniki ewaluacji eksperymentalnej

4.1. Metryki ewaluacyjne

Jakość generowania

- **ROUGE-L** - miara oparta na najdłuższym wspólnym podciągu (LCS) między wygenerowaną odpowiedzią a wzorcową. Wartości w zakresie $[0, 1]$; im wyższe tym lepiej.
- **chrF** - F-score na poziomie n-gramów znakowych, skuteczny przy krótkich odpowiedziach faktograficznych. Wartości w zakresie $[0, 100]$; im wyższe tym lepiej.

Jakość wyszukiwania

- **Recall@k** - procent pytań, dla których poprawny chunk (identyfikowany przez `question_id`) znalazł się w top- k zwróconych wynikach. Obliczany dla $k \in \{1, 3, 5\}$.

4.2. Eksperyment 1 - Wpływ RAG na jakość odpowiedzi

Pytanie badawcze: Czy dodanie retriewa i re-rankingu poprawia jakość generowanych odpowiedzi?

Oczekiwana obserwacja: $A \ll B < C < D$

Procedura: Dla każdego z 100 pytań ze zbioru ewaluacyjnego wygenerowano odpowiedzi wszystkimi czterema wariantami (A-D), a następnie porównano je z wzorcową odpowiedzią przy użyciu ROUGE-L i chrF.

Wariant	ROUGE-L	chrF
A (bez RAG)	0,0247	6,5278
B (BM25)	0,1521	20,1168
C (Dense)	0,1885	24,8143
D (Dense + Re-rank)	0,2712	32,2031

Obserwacja: $A \ll B < C \ll D$, Każde kolejne wzbogacenie pipeline'u (sparse $\rightarrow$ dense $\rightarrow$ re-ranking) daje poprawę jakości odpowiedzi.

4.3. Eksperyment 2 - Jakość wyszukiwania dokumentów

Pytanie badawcze: Która metoda wyszukiwania najlepiej odnajduje właściwy fragment?

Oczekiwane obserwacje:

- $B < C$ i D , dla wszystkich k
- $C < D$, dla małych k

Procedura: Dla każdego pytania sprawdzono, czy chunk powiązany z tym pytaniem (przez `question_id`) znalazł się w top- k wynikach wyszukiwania, dla $k \in \{1, 3, 5\}$.

Wariant	Recall@1	Recall@3	Recall@5
B (BM25)	0,6800	0,8400	0,8700
C (Dense)	0,9800	0,9800	0,9800
D (Dense + Re-rank)	0,8600	0,9800	0,9800

Obserwacje:

- $B < C$ i D , dla wszystkich k
- $C > D$, dla $k=1$

Wyszukiwanie semantyczne radzi sobie lepiej od wyszukiwania leksykalnego. Dla $k=1$ model z re-rankingiem osiąga gorszy wynik niż model z samym dense retrieval'em.

5. Podsumowanie

5.1. Osiągnięte rezultaty

W projekcie zaprojektowano, zaimplementowano i zewaluowano cztery warianty systemu QA opartego na architekturze RAG. Zbudowana baza wiedzy oparła się na 87 599 dokumentach ze zbioru SQuAD, przetworzonych na kilkudziesiąt tysięcy chunków. Pipeline integruje trzy metody wyszukiwania (BM25, ChromaDB z embeddingami BGE, re-ranking cross-encoderem) z modelem generatywnym Llama 3 8B Instruct.

Większość wyników eksperymentów jest zgodna z oczekiwaniami. Model pozbawiony dostępu do bazy dokumentów (Wariant A) nie radzi sobie z precyzyjnym odpowiadaniem na pytania faktograficzne - bez kontekstu generuje odpowiedzi ogólne lub błędne.

Wariant z wyszukiwaniem rzadkim (BM25) osiąga wyniki niewiele gorsze od wyszukiwania gęstego. Może to wynikać ze specyfiki zbioru SQuAD, w którym pytania często zawierają słowa kluczowe dosłownie obecne w odpowiednim akapicie, co sprzyja metodom leksykalnym.

Wyrażna poprawa pojawia się po zastosowaniu re-rankingu (Wariant D) - różnica względem samego wyszukiwania gęstego (Wariant C) jest większa niż zakładano. Wskazuje to, że precyzyjna selekcja najistotniejszych fragmentów istotnie ułatwia modelowi generatywnemu znalezienie właściwej odpowiedzi, nawet jeśli trafny dokument znajdował się już w zbiorze pobranym przez dense retriever.

W eksperymencie oceniającym jakość wyszukiwania (Recall@k) wyniki dla k=3 i k=5 są zgodne z oczekiwaniami: BM25 < Dense < Dense+Reranker. Wyjątkiem jest Recall@1, gdzie re-ranker osiąga wynik gorszy niż sam dense retriever. Sugeruje to, że cross-encoder nie zawsze poprawnie identyfikuje najbardziej dopasowany dokument spośród kandydatów.

5.2. Analiza krytyczna

- **Ograniczenia ewaluacji:** Recall@k opiera się na `question_id`, co zakłada, że właściwy chunk pochodzi dokładnie z akapitu powiązanego z danym pytaniem w SQuAD. W rzeczywistości podobne informacje mogą znajdować się w innych akapitach, co może zaniżać mierzone recall.
- **Metryki generowania:** ROUGE-L i chrF porównują wygenerowaną odpowiedź z jedną wzorcową odpowiedzią. W SQuAD niektóre pytania mają kilka dopuszczalnych odpowiedzi; projekt korzysta tylko z pierwszej, co może zaniżać wyniki ROUGE/chrF dla poprawnych, lecz inaczej sformułowanych odpowiedzi.
- **Kwantyzacja modelu:** Zastosowanie kwantyzacji 4-bit NF4 zmniejsza zużycie pamięci, ale może nieznacznie obniżać jakość generowania w stosunku do modelu pełnopreskrypcyjnego.

- **Rozmiar kontekstu:** Warianty B i C przekazują do 20 chunków jako kontekst, co może przekraczać optymalne okno kontekstu modelu i wprowadzać szum. Re-ranking w Wariacie D redukuje liczbę chunków do 10, co jest korzystne.

5.3. Dalsze prace

- **Hybrydowe wyszukiwanie:** Połączenie BM25 i dense retrieval (np. fuzja wyników przez Reciprocal Rank Fusion) mogłoby łączyć zalety obu metod.
- **Większa próba ewaluacyjna:** Zwiększenie liczby próbek z 100 do co najmniej 500-1000 dałoby bardziej statystycznie wiarygodne wyniki.
- **Optymalizacja parametrów:** Tuning rozmiaru chunku, zakładki, wartości top- k oraz progu re-rankingu.
- **Wariant E:** Implementacja dodatkowego wariantu, np. z fine-tuning’iem modelu tylko-dekoder na parach pytanie/odpowiedź jako dodatkowy punkt odniesienia.

Bibliografia

- [1] adr.: <https://arxiv.org/abs/2005.11401>.
- [2] adr.: <https://arxiv.org/abs/2004.04906>.
- [3] adr.: <https://arxiv.org/abs/2309.07597>.
- [4] adr.: <https://dl.acm.org/doi/10.1561/15000000019>.
- [5] adr.: <https://arxiv.org/abs/1901.04085>.
- [6] adr.: <https://arxiv.org/abs/1908.10084>.
- [7] adr.: <https://arxiv.org/abs/1606.05250>.
- [8] adr.: <https://aclanthology.org/W04-1013/>.
- [9] adr.: <https://aclanthology.org/W15-3049/>.